

Module 7 — Geant4 Sensitive Detectors

Week 7 · Hits, ROOT output, and a mini sampling calorimeter

7.1 Hits and sensitive volumes

A **sensitive detector** (SD) is a class that Geant4 calls at every step that takes place inside one of its associated logical volumes. Your SD typically creates a **hit** object storing deposited energy, position, time, track ID, and copy number. Hits accumulate in a collection attached to the event.

7.2 A minimal SD

```
class CaloHit : public G4VHit {
public:
    G4double edep{0.}, time{0.};
    G4int copyNo{-1};
};
using CaloHitsCollection = G4THitsCollection<CaloHit>;

class CaloSD : public G4VSensitiveDetector {
public:
    CaloSD(const G4String& n) : G4VSensitiveDetector(n) {
        collectionName.insert("CaloHits");
    }
    void Initialize(G4HCofThisEvent* hce) override {
        hits_ = new CaloHitsCollection(SensitiveDetectorName, collectionName[0]);
        auto id = G4SDManager::GetSDMpointer()->GetCollectionID(hits_);
        hce->AddHitsCollection(id, hits_);
    }
    G4bool ProcessHits(G4Step* st, G4TouchableHistory*) override {
        auto edep = st->GetTotalEnergyDeposit();
        if (edep <= 0) return false;
        auto h = new CaloHit;
        h->edep = edep;
        h->time = st->GetPreStepPoint()->GetGlobalTime();
        h->copyNo = st->GetPreStepPoint()->GetTouchable()->GetCopyNumber();
        hits_->insert(h);
        return true;
    }
private:
    CaloHitsCollection* hits_ = nullptr;
};
```

7.3 Attaching the SD to a volume

```
auto sd = new CaloSD("CaloSD");
G4SDManager::GetSDMpointer()->AddNewDetector(sd);
scintLV->SetSensitiveDetector(sd); // only deposits in the scintillator count
```

7.4 Writing to ROOT per event

Use a `G4UserEventAction` to grab the collection at end-of-event:

```

void EventAction::EndOfEventAction(const G4Event* e) {
    auto id = G4SDManager::GetSDMpointer()->GetCollectionID("CaloSD/CaloHits");
    auto hc = static_cast<CaloHitsCollection*>(e->GetHCofThisEvent()->GetHC(id));
    double totE = 0.;
    for (size_t i=0;i<hc->entries();++i) totE += (*hc)[i]->edep;
    anaMgr->FillNtupleDColumn(0, totE/GeV);
    anaMgr->AddNtupleRow();
}

```

Set up the analysis manager once at run-start:

```

auto am = G4AnalysisManager::Instance();
am->SetDefaultFileType("root");
am->OpenFile("calo.root");
am->CreateNtuple("calo", "Per-event totals");
am->CreateNtupleDColumn("totalEdep"); // index 0
am->FinishNtuple();

```

7.5 Macro-level control

Geant4 command macros (.mac) let you script a run without recompiling. Example run10k.mac:

```

/run/initialize
/gun/particle e-
/gun/energy 10 GeV
/gun/position 0 0 -50 cm
/gun/direction 0 0 1
/run/beamOn 10000

```

7.6 Exercises

- Build a 20-layer Pb(2 mm) / scintillator(4 mm) sandwich calorimeter and record total Edep per event for 10k electrons at 5 GeV.
- Plot the resolution $\sigma(E)/E$ vs. $1/\sqrt{E}$ for incident energies 1, 2, 5, 10, 20 GeV.
- Add a track-ID branch to the ntuple and count electromagnetic vs. hadronic tracks separately.

7.7 Self-assessment

- ☐ I have a ROOT tree with one entry per event containing total deposited energy.
- ☐ I can plot energy resolution vs. beam energy.
- ☐ I understand the difference between a **step**, **track**, **event**, and **run**.